TRABAJO FINAL DE LA ASIGNATURA DE LENGUAJES DE PROGRAMACIÓN 2024-25

VICTORIA CARBONELL SÁNCHEZ



INSPECCIÓN DE PIEZAS MEDIANTE VISIÓN ARTIFICIAL

ÍNDICE

1. Introducción 4
2. Configuración del sistema 6
   2.1. Hardware: cámara IDS, ópticas e iluminación 6
   2.2. Software: instalación del SDK y Emgu CV 6
3. Diseño de la aplicación 8
   3.1. Arquitectura general y UML básico 8
   3.2. Clase de comunicaciones con la cámara: BackEnd 11
4. Interfaz de usuario (GUI) 14
   4.1. Visualización y segmentación en tiempo real 14
   4.2. Captura y carga de imágenes 15
   4.3. Formulario de configuración de la cámara: CameraSettingsForm 16
5. Procesado y clasificación de imágenes 18
   5.1. Segmentación y extracción de características 18
   5.2. Algoritmo de clasificación de piezas 21
6. Conclusiones y mejoras futuras 22
7. Bibliografía 23

ÍNDICE DE FIGURAS

Figura 1: Montaje y Figura 2: Cámara UI-6230SE-C 4

Figura 3: Piezas mecánicas a identificar 4

Figura 4: UML de Program.cs 8

Figura 5: UML de AcquisitionWorker.cs 9

Figura 6: UML de BackEnd.cs 9

Figura 7: UML de FormWindow.cs y Figura 8: UML de CameraSettingsForm.cs 10

Figura 9: Método SetExposure() 11

Figura 10: Método SetGain() 12

Figura 11: Método SetFPS() 12

Figura 12: Interfaz gráfica de la aplicación 14

Figura 13: Visor de vídeo en tiempo real y contador de piezas 15

Figura 14: Identificación de piezas en tiempo real 15

Figura 15: Visor de imágenes y contador de piezas 16

Figura 16: CameraSettingsForm 16

Figura 17: Preprocesado de imágenes 18

Figura 18: Ejemplo de imagen binarizada de unos tornillos 19

Figura 19: Detección de contornos y jerarquía 19

Figura 20: Análisis de la jerarquía de contornos 20

Figura 21: Propiedades calculadas para cada contorno 20

Figura 22: Reglas de clasificación de piezas 21

1. Introducción

El objetivo principal del presente trabajo es el de desarrollar una aplicación software en C# que permita realizar inspecciones de pequeñas piezas mecánicas (tornillos, tuercas, arandelas, etc.) mediante técnicas de visión artificial. Para ello, se utiliza una cámara industrial IDS modelo "UI-6230SE-C" junto con un sistema de iluminación, ópticas intercambiables de diferente distancia focal y un trípode de sujeción.

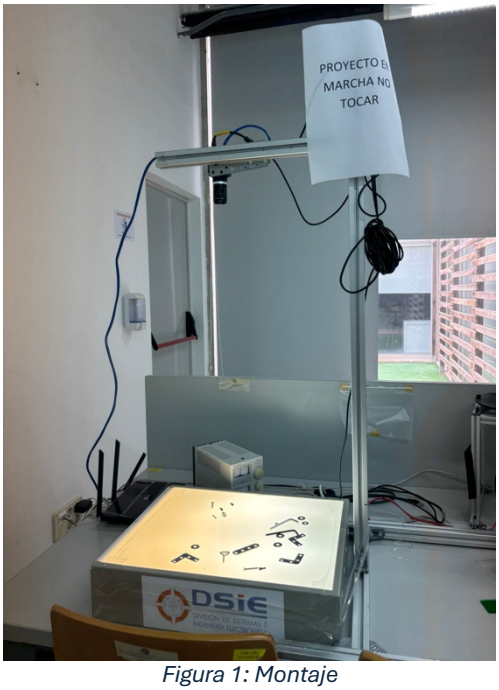


Figura 1: Montaje

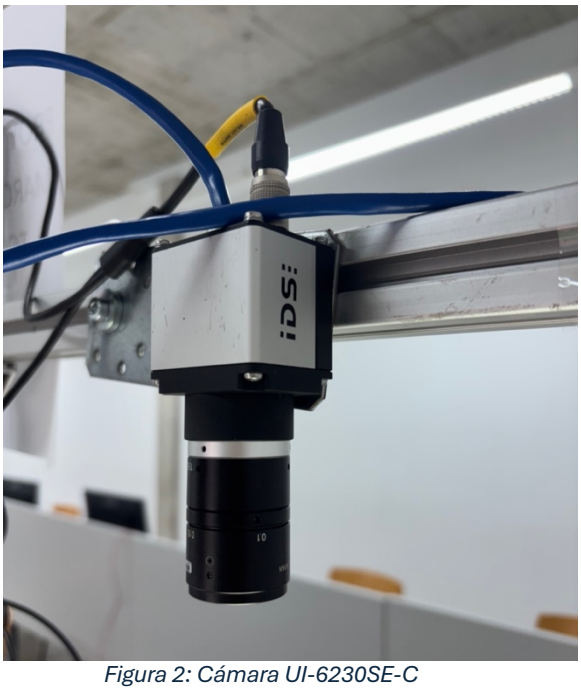


Figura 2: Cámara UI-6230SE-C

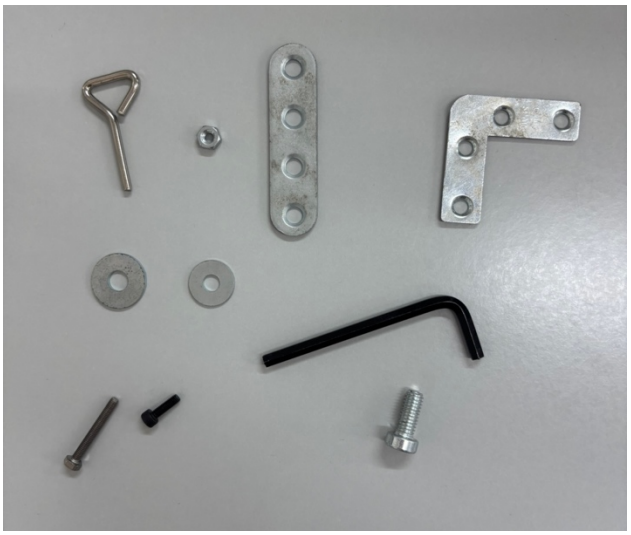


Figura 3: Piezas mecánicas a identificar

La aplicación desarrollada deberá cumplir con los siguientes requisitos funcionales mínimos:

- Captura de imágenes en tiempo real mediante la cámara industrial.
- Segmentación de objetos en tiempo de vídeo.
- Extracción de las características de los objetos capturados y clasificación automática de al menos tres tipos de piezas.
- Interfaz gráfica de usuario (GUI) que permita interactuar con la cámara (configuración de parámetros como la ganancia o la exposición), visualizar imágenes y gestionar el proceso de inspección de las piezas.

El alcance del trabajo abarca desde la configuración inicial del sistema (hardware y software) hasta la implementación de la aplicación y su validación mediante pruebas con imágenes reales. Todo ello incluye la familiarización con el uso de cámaras industriales y el uso de librerías de visión por computador como Emgu CV, la versión en C# de OpenCV.

2. Configuración del sistema

2.1. Hardware: cámara IDS, ópticas e iluminación

Para llevar a cabo la inspección de las piezas presentadas en el capítulo anterior, se ha empleado la cámara IDS modelo "UI-6230SE-C", cuyas especificaciones principales son, según el manual del fabricante [1]:

- **Sensor:** El corazón de la cámara es un sensor CCD Color Sony ICX204AK de 0,79 Mpx (1024 x 768 píxeles), con píxeles de 4,65 µm y obturador global. Dispone de un ADC (Analog-to-Digital Converter) de 12 bits, que ofrece salida a 12 bit y una ganancia de hasta 7,6x en modo máster (4x en canal RGB).
- **Rendimiento de captura:** Permite adquirir hasta 40 fps en modos freerun y en trigger, con un rango del reloj de píxeles entre 10 MHz y 40 MHz. El tiempo de exposición se puede variar entre 0,023 ms y 120000 ms, y cuenta con 60MB de memoria onboard y función overlap trigger para optimizar la adquisición.
- **Conectividad y alimentación:** La transmisión de datos se realiza por GigE (RJ45 con rosca). Además, dispone de un conector I/O Hirose de 6 pines que ofrece 1 entrada de disparo por octoacoplador y 1 salida de flash, también por octoacoplador. El equipo se alimenta con 12 V – 24 V DC.
- **Montura y carcasa:** La cámara posee una montura de objetivo C-Mount. La carcasa de aluminio tiene protección IP30 y unas dimensiones de 34,0 mm x 44,0 mm x 56,8 mm (altura x anchura x longitud).
- **Condiciones ambientales:** Las condiciones de operación de la cámara son de 0ºC a 55ºC. Mientras que, de almacenamiento, son de -20ºC a 60ºC. El dispositivo soporta una humedad relativa, sin condensación de entre el 20 % y el 80 %.

Para seleccionar la lente adecuada se ha tenido en cuenta el tamaño de las piezas a inspeccionar, así como la distancia de trabajo disponible. Se ha optado por una lente de 16 mm de distancia focal, apertura del diafragma de 1:1,4 y un tamaño máximo del sensor de 2/3" (8,8 mm x 6,6 mm aprox.). También tiene una rosca tipo C, compatible con la cámara IDS que se está utilizando.

Para resaltar la silueta de las piezas y facilitar su identificación se ha usado un sistema de iluminación a contraluz. Las piezas se colocan sobre un panel que emite luz desde abajo, consiguiendo un contraste elevado entre la pieza (en sombra) y el fondo luminoso. Esto simplifica considerablemente la binarización de la imagen y la obtención de un perfil nítido de cada objeto que permite extraer fácilmente su contorno.

2.2. Software: instalación del SDK y Emgu CV

Para poder comunicarse con la cámara IDS, ha sido necesario instalar y configurar el uEye SDK (Software Development Kit) oficial de IDS, aparte de la librería Emgu CV (wrapper de OpenCV para .NET).

Como requisitos previos cabe mencionar el IDE Visual Studio 2022 con carga de trabajo .NET Desktop Development (framework .NET 6). El software de IDS se ha descargado desde

la página web de IDS Imaging[1]. Tras la instalación y posterior conexión de la cámara al PC, se ha verificado el correcto funcionamiento de la cámara mediante la aplicación IDS Peak Cockpit, incluida en el SDK, comprobando que la cámara aparece listada y es capaz de mostrar imagen en tiempo real.

Por otro lado, a través de la aplicación IDS uEye Manager se ha configurado la interfaz de red del ordenador con IP fija en el mismo rango que la cámara para evitar que se pierda la conexión:

- PC: 192.168.11.117
- Cámara: 192.168.0.150
- Máscara de subred: 255.255.255.0
    - Inicio de rango IP: 192.168.11.1
    - Fin de rango IP: 192.168.11.250

En cuanto a la instalación de Emgu CV, esta se ha realizado a través del Administrador de paquetes NuGet, accesible desde la pestaña de “Herramientas” de Visual Studio. Se han descargado las siguientes dependencias, verificando la compatibilidad de versiones entre ellas y con .NET 6:

- Emgu.CV v4.5.5.4823
- Emgu.CV.runtime.windows v4.5.5.4823
- Emgu.CV.UI v4.5.5.4823
- Emgu.CV.Bitmap v4.5.5.4823
- System.Drawing.Common v6.0.0

Antes de comenzar a programar la aplicación se ha realizado una prueba inicial de funcionamiento con uno de los ejemplos de código que IDS proporciona al descargar el SDK (“simple_live_windows_forms”). Se trata de un pequeño proyecto de Windows Forms Application en C# que consiste en una interfaz de usuario que muestra vídeo en tiempo real.

Una vez asegurado el correcto funcionamiento de la cámara, se ha usado este ejemplo de código como base para empezar a desarrollar la aplicación, pues ya contenía una clase, denominada BackEnd, con las funciones básicas para inicializar y cerrar la cámara, iniciar y detener la captura, y transmitir cada frame a la interfaz. Estas funciones se comentarán con mayor profundidad en apartados posteriores.

[1] https://es.ids-imaging.com

3. Diseño de la aplicación

3.1. Arquitectura general y UML básico

La estructura de carpetas del proyecto es, a grandes rasgos, la siguiente:

camara_VCS:

- AcquisitionWorker.cs
- app.config
- **BackEnd.cs**
- bin
- **CameraSettingsForm.cs**
- **FormWindow.cs**
- **FormWindow.Designer.cs**
- FormWindow.resx
- obj
- Program.cs
- Properties
- simple_live_windows_forms_csharp_framework.csproj
- simple_live_windows_forms_csharp_framework.v12.suo
- **simple_live_windows_forms_csharp_VCS.csproj**
- simple_live_windows_forms_csharp_VCS.csproj.user
- simple_live_windows_forms_csharp_VCS.sln
- simple_live_windows_forms_csharp.csproj.user
- simple_live_windows_forms_csharp.sln
- SimplePictureBox.cs

Todo lo anterior forma parte del ejemplo que ofrece IDS, y se han resaltado en negrita los archivos que se han modificado para realizar el presente proyecto.

Los siguientes diagramas UML muestran las clases principales definidas en el namespace “simple_live_windows_forms”:

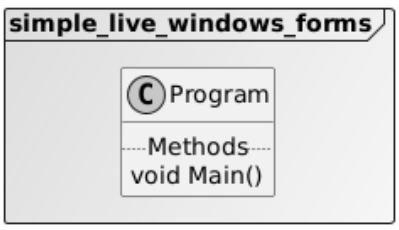


Figura 4: UML de Program.cs

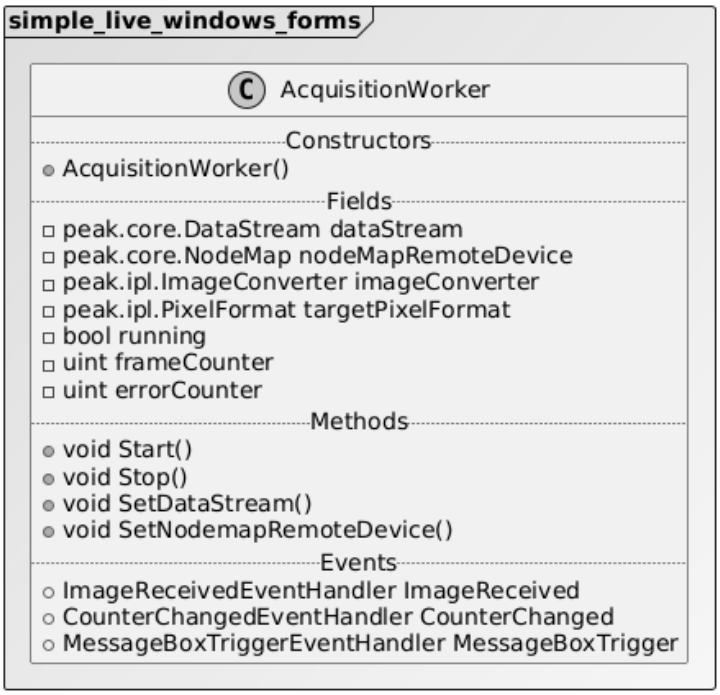


Figura 5: UML de AcquisitionWorker.cs



Figura 6: UML de BackEnd.cs



Figura 7: UML de FormWindow.cs



Figura 8: UML de CameraSettingsForm.cs

3.2. Clase de comunicaciones con la cámara: BackEnd

Para cumplir con el requisito de disponer de una capa de comunicaciones, se ha reutilizado y ampliado la clase pública BackEnd, incluida en el ejemplo oficial de IDS, que ya implementaba los métodos básicos Start(), Stop(), OpenDevice() y CloseDevice(), aparte de su correspondiente constructor, BackEnd().

A esta clase se le han añadido una serie de métodos que permiten ajustar en tiempo real los parámetros de adquisición de la cámara:

- SetExposure() para establecer la exposición de la cámara en milisegundos:

```
// Método para establecer la exposición de la cámara en milisegundos
public void SetExposure(double valorMs)
{
    try
    {
        // Buscar el nodo “ExposureTime”
        var nodoExpo = nodeMapRemoteDevice.FindNode<FloatNode>("ExposureTime");

        if (nodoExpo != null && nodoExpo.IsWriteable())
        {
            // Convertimos valorMs (milisegundos) a microsegundos:
            double valorUs = valorMs * 1000.0;

            double minUs = Convert.ToDouble(nodoExpo.Minimum());
            double maxUs = Convert.ToDouble(nodoExpo.Maximum());

            // Recortamos al rango válido de exposición
            double nuevoUs = Math.Max(minUs, Math.Min(maxUs, valorUs));

            // Establecemos el nuevo valor de exposición
            nodoExpo.SetValue(nuevoUs);
        }
    }
    catch
    {
        Debug.WriteLine("--- [BackEnd] Error al establecer la exposición");
    }
}
```

Figura 9: Método SetExposure()

- SetGain() para modificar la ganancia:

```
// Método para establecer la ganancia de la cámara
public void SetGain(double valor)
{
    try
    {
        // Buscar el nodo “Gain”
        var nodoGanancia = nodeMapRemoteDevice.FindNode<FloatNode>("Gain");

        if (nodoGanancia != null && nodoGanancia.IsWriteable())
        {
            double min = nodoGanancia.Minimum();
            double max = nodoGanancia.Maximum();
            double nuevo = Math.Max(min, Math.Min(max, valor));

            // Establecemos el nuevo valor de ganancia
            nodoGanancia.SetValue(nuevo);
        }
    }
    catch
    {
        Debug.WriteLine("--- [BackEnd] Error al establecer la ganancia");
    }
}
```

Figura 10: Método SetGain()

- SetFPS() para elegir los fotogramas por segundo que se adquieren:

```
// Método para establecer los FPS de la cámara
public void SetFPS(double fps)
{
    try
    {
        // Nodo “AcquisitionFrameRate”
        var nodoRate = nodeMapRemoteDevice.FindNode<FloatNode>("AcquisitionFrame

        if (nodoRate != null && nodoRate.IsWriteable())
        {
            double min = Convert.ToDouble(nodoRate.Minimum());
            double max = Convert.ToDouble(nodoRate.Maximum());
            double nuevo = Math.Max(min, Math.Min(max, fps));

            // Establecemos el nuevo valor de FPS
            nodoRate.SetValue(nuevo);
        }
    }
    catch
    {
        Debug.WriteLine("--- [BackEnd] Error al establecer los FPS");
    }
}
```

Figura 11: Método SetFPS()

Estos tres métodos siguen un patrón común:

1. **Localización del nodo:** Se invoca nodeMapRemoteDevice.FindNode<Nodo>(“NombreDelParametro”) para obtener el objeto que representa el parámetro que se desea modificar en el árbol de nodos del SDK.
2. **Verificación de escritura:** Se comprueba que el nodo existe y que se puede modificar con nodo.IsWriteable().
3. **Clamping del valor:** Se obtienen los valores máximo y mínimo de dicho nodo y se recorta el valor de entrada de modo que quede dentro de ese rango.
4. **Aplicación del parámetro**: Se asigna el nuevo valor al nodo mediante nodo.SetValue(nuevo).
5. **Manejo de errores**: Cada método contiene un bloque try-catch que captura las excepciones y emite un mensaje de error en caso de fallo.

4. Interfaz de usuario (GUI)

En este capítulo se describe el diseño e implementación de la interfaz gráfica de usuario de la aplicación. El objetivo de la GUI es ofrecer al usuario un entorno intuitivo desde el que visualizar en tiempo real el vídeo capturado (con la posibilidad de ver superpuestos los resultados de la segmentación y clasificación de piezas), capturar y/o cargar imágenes desde el ordenador e identificar las piezas mecánicas en ellas, y configurar los parámetros de la cámara.

La GUI se ha desarrollado usando Windows Forms en C#, aprovechando los controles estándar como botones, trackbars o picture boxes. A continuación, se muestra una captura de la ventana principal de la aplicación, donde se aprecian el visor de vídeo y las leyendas de clasificación.

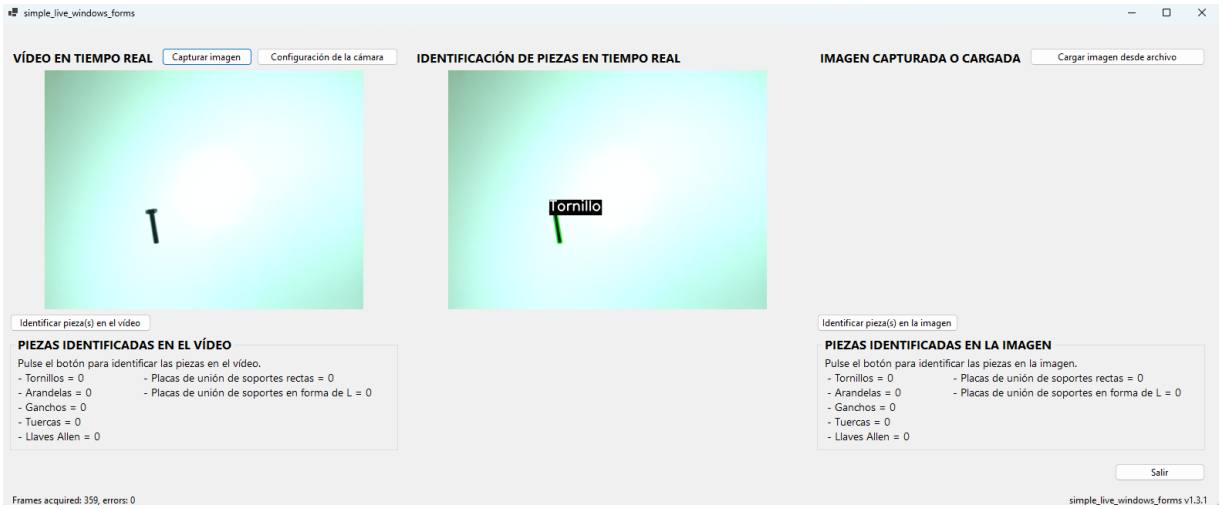


Figura 12: Interfaz gráfica de la aplicación

En las siguientes secciones se detallan cada uno de los elementos de la interfaz, su disposición, las acciones que desencadenan en el backend y cómo se actualizan en tiempo real para reflejar el estado de la inspección de piezas.

4.1. Visualización y segmentación en tiempo real

La columna izquierda de la ventana principal está dedicada a la visualización en tiempo real del flujo de vídeo procedente de la cámara. Bajo el visor, el botón “Identificar pieza(s) en el vídeo” permite analizar el último frame capturado, y mostrar en el contador de la esquina inferior izquierda cuántas piezas de cada tipo se han identificado. De este modo el usuario puede obtener un conteo rápido de cada categoría.



Figura 13: Visor de vídeo en tiempo real y contador de piezas

Por otro lado, la columna central de la interfaz analiza el frame actual, segmenta las regiones de interés y asigna la etiqueta correspondiente a cada pieza.



Figura 14: Identificación de piezas en tiempo real

4.2. Captura y carga de imágenes

La columna de la derecha sirve para trabajar con imágenes estáticas. Desde el botón “Capturar imagen” de la Figura 13, se puede tomar el frame actual de la columna de vídeo de la izquierda y mostrarlo en un panel independiente para su análisis. También es posible, desde el botón “Cargar imagen desde archivo”, abrir un cuadro de diálogo para importar cualquier imagen compatible.

Al pulsar el botón “Identificar pieza(s) en la imagen”, una vez que la imagen esté cargada o capturada, se aplica el mismo mecanismo de segmentación y clasificación de piezas, pero sobre la imagen fija. Las piezas detectadas aparecen etiquetadas con su nombre en la imagen, y también se registran en el contador de la esquina inferior derecha. Esto facilita el análisis de imágenes concretas, sin necesidad de conexión con la cámara.



Figura 15: Visor de imágenes y contador de piezas

4.3. Formulario de configuración de la cámara: CameraSettingsForm

El botón “Configuración de la cámara” ubicado en la parte superior del visor de vídeo en tiempo real abre una GUI secundaria (CameraSettingsForm) que agrupa los controles necesarios para ajustar parámetros tales como la exposición, la ganancia y los fps de la cámara.

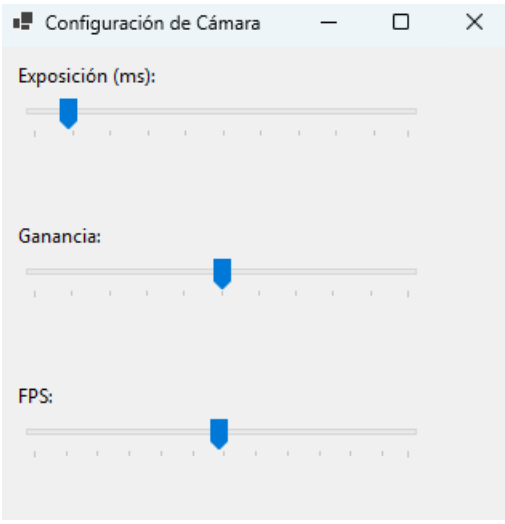


Figura 16: CameraSettingsForm

- **Trackbar Exposición (ms):** Permite variar el tiempo de exposición entre el mínimo y el máximo admitidos por la cámara. A medida que el usuario mueve el deslizador, el valor numérico de la exposición (en milisegundos) se actualiza en un label adyacente y se envía al BackEnd.
- **Trackbar Ganancia:** Controla el nivel de amplificación de la señal eléctrica que proviene del sensor de la cámara. Aumentar la ganancia hace que la imagen se vea más brillante, pero también puede introducir ruido en la imagen.
- **Trackbar FPS:** Ajusta la tasa de adquisición de frames por segundo. Mover este deslizador modifica la velocidad a la que el BackEnd solicita nuevos frames.

5. Procesado y clasificación de imágenes

A continuación, se van a describir las técnicas empleadas para segmentar las piezas en la imagen, extraer sus características más relevantes y asignarles una etiqueta. Esta técnica se ha utilizado tanto en el vídeo en tiempo real como en las imágenes estáticas.

5.1. Segmentación y extracción de características

1. Preprocesado

En primer lugar, se captura la imagen de cámara y se asigna al picture box. Después, esta imagen se convierte a bitmap. Luego, el bitmap se convierte a una imagen a color (BGR), se pasa a escala de grises para simplificar el análisis.

Dado que se trabaja a contraluz se utiliza un umbral (threshold) para separar el fondo luminoso de las piezas en sombra. El resultado es una imagen binarizada donde los píxeles blancos (valor 1) representan las piezas y los negros (valor 0) se corresponden con el fondo.

```
// Capturar imagen desde la cámara y asignarla al PictureBox
try
{
    if (pictureBox.Image != null)
    {
        // Convertir la imagen del PictureBox a Bitmap
        Bitmap bmp = new Bitmap(pictureBox.Image);

        // Convertir Bitmap a Image<Bgr, byte>
        ImageBGR = bmp.ToImage<Bgr, byte>();

        // Crear contenedor para la imagen en escala de grises
        ImageGray = ImageBGR.Convert<Gray, byte>();

        // Crear contenedor para la imagen binaria
        ImageBin = ImageGray.ThresholdBinaryInv(new Gray(100), new Gray(255));

        // Mostrar la imagen binaria en el segundo PictureBox
        pbtImage.Image = ImageBin.ToBitmap();
    }
    else
    {
        MessageBox.Show("No hay imagen en el PictureBox.");
    }
}
catch (Exception ex)
{
    Debug.WriteLine("Error al capturar la imagen desde la cámara: " + ex.Message);
}
```

Figura 17: Preprocesado de imágenes

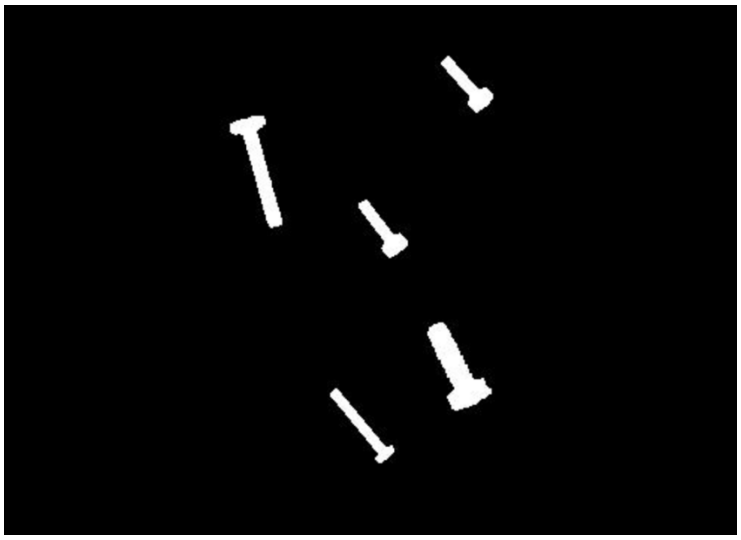


Figura 18: Ejemplo de imagen binarizada de unos tornillos

2. Detección de contornos

Con CvInvoke.FindContours se analiza la imagen binaria para obtener los diferentes contornos (piezas) que hay en ella. Estos se almacenan en un vector de vectores de puntos (VectorOfVectorOfPoint), que se usa para almacenar un conjunto de puntos, cada uno definido por sus coordenadas. Además, la jerarquía de contornos se almacena en una matriz, lo que luego permitirá diferenciar los contornos "padre" (piezas) de los "hijos" (agujeros).

```
// Encontrar las piezas (contornos) y la jerarquía en la imagen binaria
VectorOfVectorOfPoint contornos = new VectorOfVectorOfPoint();
Mat jerarquia = new Mat(); // Matriz para almacenar la jerarquía de los contornos
CvInvoke.FindContours(
    ImageBin,
    contornos,
    jerarquia,
    Emgu.CV.CvEnum.RetrType.Tree,
    Emgu.CV.CvEnum.ChainApproxMethod.ChainApproxSimple);
```

Figura 19: Detección de contornos y jerarquía

3. Extracción de características

Por último, se comprueba para cada contorno si se trata de una pieza o de un agujero, en cuyo caso, se descarta.

```
// Analizar cada contorno
for (int i = 0; i < contornos.Size; i++)
{
    // Comprobar si el contorno tiene agujeros
    if (jerarquiaDatos != null)
    {
        int padre = jerarquiaDatos[i * 4 + 3]; // Índice del contorno padre

        // Si este contorno es padre
        if (padre == -1)
        {
            // Contar los contornos de dentro
            numAgujeros = jerarquiaDatos.Where((p, j) => ((j + 1) % 4 == 0) && p == i).Count();
        }
        else
        {
            continue; // Si es un contorno hijo, lo ignoramos
        }
    }

    // Obtener el contorno externo i
    var contorno = contornos[i];
```

Figura 20: Análisis de la jerarquía de contornos

Para cada contorno válido (“padre”) se calculan las siguientes características:

- Número de agujeros (ver Figura 20)
- Área
- Perímetro
- Rectángulo delimitador y relación de aspecto
- Circularidad
- Solidez
- Número de vértices, aproximando el contorno a un polígono
- Convexidad del polígono aproximado

```
// Propiedades del contorno
double area = CvInvoke.ContourArea(contorno); // Área del contorno
double perimetro = CvInvoke.ArcLength(contorno, true); // Perímetro del contorno
Rectangle rectanguloDelimitador = CvInvoke.BoundingRectangle(contorno); // Rectángulo que
                                                                        // encierra completamente el contorno

double relacionAspecto = (double)rectanguloDelimitador.Width / rectanguloDelimitador.Height; // Relación entre el
                                                                                             // ancho y el alto del
                                                                                             // rectángulo delimitador
double circularidad = 4 * Math.PI * area / (perimetro * perimetro); // Circularidad = 1 para un círculo perfecto
double solidez = area / (rectanguloDelimitador.Width * rectanguloDelimitador.Height); // Solidez = relación
                                                                                      // entre el área del contorno
                                                                                      // y el área del rectángulo
                                                                                      // delimitador


// Aproximar el contorno a un polígono
double epsilon = 0.02 * perimetro; // Epsilon es un parámetro que controla la precisión de la aproximación
var aproximacionPoligono = new VectorOfPoint();
CvInvoke.ApproxPolyDP(contorno, aproximacionPoligono, epsilon, true);

// Contar el número de vértices del polígono aproximado
int numVertices = aproximacionPoligono.Size;

// Determinar si el polígono es convexo o cóncavo
bool esConvexo = CvInvoke.IsContourConvex(aproximacionPoligono); // true si es convexo (no tiene forma de L)
```

Figura 21: Propiedades calculadas para cada contorno

5.2. Algoritmo de clasificación de piezas

Una vez que se dispone de las características de cada pieza, estas se clasifican según las reglas que se presentan en la siguiente tabla:

	Nº agujeros	Área	Perímetro	Relación aspecto	Circularidad	Solidez	Nº vértices	Convexo
Arandela	== 1			> 0,75 y < 1,25	> 0,75		> 7	
Tuerca	== 1						>= 5 y <= 7	
Placa unión soportes en L	> 1	> 14000						0
Placa unión soportes recta	> 1	> 14000						1
Tornillo		< 3000			< 0,5			
Llave Allen	== 0	> 5000				< 0,25		0
Gancho	== 0	> 3000 y < 5000					> 7	0

Figura 22: Reglas de clasificación de piezas

Para presentar los resultados, sobre las imágenes se superpone un recuadro con el nombre de cada pieza y, en las listas, se incrementa el contador correspondiente a cada pieza.

6. Conclusiones y mejoras futuras

La aplicación desarrollada cumple con los requisitos iniciales: captura en tiempo real, segmentación fiable, extracción de características y clasificación automática de al menos tres tipos de piezas distintas. La interfaz de usuario resulta intuitiva y permite tanto el análisis en directo como el procesamiento de imágenes estáticas.

Mejoras propuestas para trabajos futuros:

- **Guardado de imágenes etiquetadas:** Implementar la capacidad de almacenar los fotogramas con los contornos y etiquetas dibujados.
- **Ampliar los parámetros configurables:** Incluir en CameraSettingsForms el control de otros ajustes: contraste, blanco y negro / color...
- **Restauración de la configuración:** Añadir un botón de “Restablecer” dentro de CameraSettingsForm que devuelva todos los parámetros de la cámara a su estado original.
- **Gestión de solapamiento:** Desarrollar una función, por ejemplo, con Watershed, para separar las piezas superpuestas y mejorar la precisión de la aplicación en casos de solapamiento.

7. Bibliografía

[1] IDS Imaging Development Systems Inc., “UI-6230SE-C-HQ_Rev_3”, Accessed: Jun. 09, 2025. [Online]. Available: www.ids-imaging.us